DAY 9

Advanced JavaScript Topics — Notes, Programs & Practical Project

Learning objectives

• Understand DOM manipulation and event handling.

• Practice map(), filter(), and reduce().

• Use ES6 features, arrow functions, destructuring, spread, and rest.

• Understand Promises, async/await, and Fetch API basics.

• Build an interactive Student Dashboard webpage.

1. DOM Manipulation

DOM (Document Object Model) lets JavaScript access and modify HTML elements.

```
<h1 id="heading">Hello World</h1>
<button onclick="changeText()">Click Me</button>

<script>
function changeText() {
    document.getElementById("heading").textContent =
        "Welcome to JavaScript!";
}
</script>
```

When the button is clicked, the heading changes to “Welcome to JavaScript!”

Useful DOM methods

Method / Property	Purpose
getElementById()	Select an element by ID.
querySelector()	Select the first matching CSS selector.
querySelectorAll()	Select all matching elements.
textContent	Read or change text.
innerHTML	Read or change HTML content; avoid inserting untrusted input.
style	Change CSS styles.
createElement()	Create a new HTML element.
appendChild()	Append an element to another element.

2. Event Handling

Button-click event

```
<button id="myButton">Click Me</button>
<p id="message"></p>

<script>
const button = document.getElementById("myButton");
const message = document.getElementById("message");

button.addEventListener("click", function () {
    message.textContent = "Button clicked successfully!";
});
</script>
```

Form-submission event

```
<form id="myForm">
    <input type="text" id="username"
           placeholder="Enter your name" required>
    <button type="submit">Submit</button>
</form>
<p id="result"></p>

<script>
const form = document.getElementById("myForm");

form.addEventListener("submit", function (event) {
    event.preventDefault();
    const name = document.getElementById("username").value;
    document.getElementById("result").textContent =
        "Welcome, " + name;
});
</script>
```

Key ideas: addEventListener() listens for an event; preventDefault() prevents the form's normal page reload; .value reads an input's value.

3. Array Methods: map(), filter(), reduce()

map() — transform every element

```
const numbers = [1, 2, 3, 4, 5];
const squares = numbers.map(num => num * num);
console.log(squares);
// [1, 4, 9, 16, 25]
```

filter() — select matching elements

```
const numbers = [10, 15, 20, 25, 30];
const result = numbers.filter(num => num > 20);
console.log(result);
// [25, 30]
```

reduce() — combine values into an accumulated result

```
const numbers = [10, 20, 30, 40];
const total = numbers.reduce((sum, num) => sum + num, 0);
console.log(total);
// 100
```

Quick comparison: map() returns a transformed array; filter() returns matching elements; reduce() accumulates values into a result.

4. ES6 Features

let and const

```
let age = 22;
age = 23;
const name = "Vaibhav";

console.log(age);   // 23
console.log(name);  // Vaibhav
```

let can be reassigned. const cannot be reassigned, but objects and arrays declared with const can still have their contents changed.

Template literals

```
const name = "Vaibhav";
const age = 22;
console.log(`My name is ${name} and I am ${age} years old.`);
// My name is Vaibhav and I am 22 years old.
```

5. Arrow Functions

```
function add(a, b) {
    return a + b;
}

const addArrow = (a, b) => a + b;

console.log(add(10, 20));      // 30
console.log(addArrow(5, 3));   // 8
```

Arrow functions provide concise syntax. They also handle `this` differently from regular functions.

6. Destructuring

Array destructuring

```
const colors = ["Red", "Green", "Blue"];
const [first, second, third] = colors;

console.log(first);   // Red
console.log(second);  // Green
console.log(third);   // Blue
```

Object destructuring

```
const student = {
    name: "Vaibhav",
    age: 22,
    course: "JavaScript"
};

const { name, age, course } = student;
console.log(name);    // Vaibhav
console.log(age);     // 22
console.log(course);  // JavaScript
```

7. Spread and Rest Operators

Spread — expand array values

```
const first = [10, 20, 30];
const second = [40, 50];
const combined = [...first, ...second];

console.log(combined);
// [10, 20, 30, 40, 50]
```

Rest — collect arguments

```
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(10, 20, 30));     // 60
console.log(sum(1, 2, 3, 4, 5)); // 15
```

Spread expands values; rest collects multiple values into an array.

8. Promises and Asynchronous JavaScript

A Promise represents the eventual success or failure of an asynchronous operation. Its states are pending, fulfilled, and rejected.

```
const myPromise = new Promise((resolve, reject) => {
    const success = true;

    if (success) {
        resolve("Operation successful!");
    } else {
        reject("Operation failed!");
    }
});

myPromise
    .then(result => console.log(result))
    .catch(error => console.log(error));

// Output: Operation successful!
```

9. async and await

```
function getData() {
    return new Promise(resolve => {
        setTimeout(() => {
            resolve("Data received successfully!");
        }, 2000);
    });
}

async function displayData() {
    console.log("Loading...");
    const result = await getData();
    console.log(result);
}

displayData();

// Loading...
// (about 2 seconds later) Data received successfully!
```

await pauses the current async function while the Promise settles; it does not block the JavaScript main thread.

10. Fetch API Basics

```
async function getUsers() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
            throw new Error("HTTP error: " + response.status);
        }

        const users = await response.json();
        console.log(users);
    } catch (error) {
        console.log("Error:", error.message);
    }
}

getUsers();
```

Flow: fetch(url) sends a request → response.ok checks HTTP status → response.json() parses JSON → catch handles network or processing errors. Fetch does not automatically reject for HTTP 404/500 responses, so check response.ok.

11. Practical Project — Interactive Student Dashboard

Create a folder named **Day9** with two files: `index.html` and `script.js`.

File 1: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Dashboard</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 750px;
            margin: 30px auto;
            padding: 20px;
            background: #f4f6f9;
        }
        h1 { text-align: center; }
        form, .controls, .student-card {
            background: white;
            padding: 15px;
            margin-bottom: 15px;
            border-radius: 8px;
        }
        input, button { padding: 10px; margin: 5px 0; }
        input { box-sizing: border-box; max-width: 100%; }
        form input { width: 100%; }
        button { cursor: pointer; }
        .student-card { border: 1px solid #ddd; }
        #message { font-weight: bold; }
    </style>
</head>
<body>
    <h1>Student Dashboard</h1>

    <form id="studentForm">
        <h2>Add Student</h2>
        <input type="text" id="name"
               placeholder="Student name" required>
        <input type="number" id="marks"
               placeholder="Marks (0-100)"
               min="0" max="100" required>
        <button type="submit">Add Student</button>
    </form>

    <div class="controls">
        <h2>Search Students</h2>
        <input type="text" id="search" placeholder="Search by name">
        <button id="showAll">Show All</button>
        <button id="showPassed">Show Passed (40+)</button>
        <button id="loadUsers">Load API Users</button>
        <p id="message" aria-live="polite"></p>
    </div>

    <h2>Students</h2>
    <div id="studentList"></div>

    <script src="script.js"></script>
</body>
</html>
```

File 2: script.js

```
let students = [
    { id: 1, name: "Rahul", marks: 75 },
    { id: 2, name: "Priya", marks: 85 },
    { id: 3, name: "Amit", marks: 32 },
    { id: 4, name: "Sneha", marks: 90 }
];

const studentList = document.getElementById("studentList");
const studentForm = document.getElementById("studentForm");
const searchInput = document.getElementById("search");
const message = document.getElementById("message");

const displayStudents = (data) => {
    studentList.replaceChildren();

    if (data.length === 0) {
        studentList.textContent = "No students found.";
        return;
    }

    data.forEach(({ name, marks }) => {
        const card = document.createElement("div");
        card.className = "student-card";

        const title = document.createElement("h3");
        title.textContent = name;

        const score = document.createElement("p");
        score.textContent = `Marks: ${marks}`;

        const status = document.createElement("p");
        status.textContent = marks >= 40 ? "Status: Pass" : "Status: Fail";

        card.append(title, score, status);
        studentList.appendChild(card);
    });
};

document.getElementById("showAll").addEventListener("click", () => {
    displayStudents(students);
    message.textContent = "Showing all students.";
});

document.getElementById("showPassed").addEventListener("click", () => {
    const passed = students.filter(student => student.marks >= 40);
    displayStudents(passed);
    message.textContent = `Passed students: ${passed.length}`;
});

searchInput.addEventListener("input", () => {
    const searchText = searchInput.value.toLowerCase().trim();

    const filteredStudents = students.filter(student =>
        student.name.toLowerCase().includes(searchText)
    );

    displayStudents(filteredStudents);
});

studentForm.addEventListener("submit", event => {
    event.preventDefault();

    const name = document.getElementById("name").value.trim();
    const marks = Number(document.getElementById("marks").value);

    if (!name || !Number.isFinite(marks) || marks < 0 || marks > 100) {
        message.textContent = "Enter a valid name and marks between 0 and 100.";
        return;
    }

    const newStudent = {
        id: Date.now(),
        name,
        marks
```

```
    };

    students = [...students, newStudent];
    displayStudents(students);
    studentForm.reset();
    message.textContent = `${name} added successfully!`;
});

document.getElementById("loadUsers").addEventListener("click", async () => {
    message.textContent = "Loading users...";

    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`);
        }

        const users = await response.json();

        const apiStudents = users.map(({ id, name }) => ({
            id,
            name,
            marks: 0
        }));

        students = [...students, ...apiStudents];
        displayStudents(students);
        message.textContent = `${users.length} API users loaded.`;
    } catch (error) {
        message.textContent = `Failed to load users: ${error.message}`;
    }
});

displayStudents(students);
```

12. How to Run the Project

• Save both files using Ctrl + S.

• If Live Server is installed in VS Code, right-click index.html and select “Open with Live Server”.

• Use the webpage buttons to add students, search names, filter passed students, and fetch API users.

• An internet connection is required for the Fetch API button.

Note: Student data is held in memory only. Refreshing the page resets added students and API data.

13. Extra Practice Tasks

Task	Requirement	Concept
1	Filter students who failed.	filter()
2	Add 5 marks to each student's marks.	map()
3	Calculate average marks.	reduce()
4	Find the student with the highest marks.	Arrays and conditions
5	Add a delete button for each student.	DOM and events
6	Show a loading indicator while fetching users.	async/await
7	Sort students by marks, highest first.	sort()

Example: Average marks

```
const totalMarks = students.reduce(
    (total, student) => total + student.marks,
    0
);

const average = students.length
    ? totalMarks / students.length
    : 0;

console.log("Average Marks:", average);
// Initial students: 70.5
```

14. Day 9 Completion Checklist

- ■ DOM manipulation understood.
- ■ Button and form events implemented.
- ■ map(), filter(), and reduce() practiced.
- ■ Arrow functions and destructuring used.
- ■ Spread operator used to combine arrays.
- ■ Promises and async/await basics understood.
- ■ Fetch API used to retrieve data.
- ■ Interactive Student Dashboard run successfully.

Recommended order: DOM → Events → Array Methods → ES6 → Promises → Fetch API → Project.